

Proving CGGI Bootstrapping with a Lattice Sumcheck Argument: an Expository Note

June 2026

Abstract

This note explains, at an introductory level, how a CGGI (TFHE) programmable bootstrapping is proven correct inside a lattice-based succinct argument. The construction has three parts. First, the bootstrapping itself is instantiated natively over the prime modulus $q = 2^{50} - 2687$ of the proof system, with the parameter shape of the TFHE-rs default set ($n = 918$ blind-rotation steps, ring degree $N = 2048$); one bootstrapping then runs in 0.3 seconds in software. Second, every polynomial of the execution trace is committed in a packed evaluation domain in which a product of TFHE-ring polynomials becomes a short convolution of products in the proof ring; this is what turns the 918 ciphertext multiplications of a blind rotation into constraints a sumcheck can process. Third, the whole trace is expressed as a handful of functional sumcheck claims over one committed witness: linear claims for gadget recomposition and the public boundary values, copy claims that align multiplication operands, and one degree-two claim covering all multiplication steps at once. A bootstrapping at these full parameters is proven end to end through the argument chain, including a random-projection round that binds a strict ℓ_2 norm of the committed trace: the prover runs in 11.6 minutes, dominated by the entry sumcheck, and the proof is 167 KB. At toy parameters, four independent bootstrappings under one committed key prove in under three minutes and verify in 51 milliseconds, with the same proof size. The note assumes no prior familiarity with either TFHE or sumcheck arguments.

1 Introduction

Fully homomorphic encryption schemes of the CGGI family [1] evaluate functions on encrypted data by *bootstrapping*: a public-key operation that simultaneously refreshes the noise of a ciphertext and applies a lookup table to the encrypted value. A natural integrity question arises whenever an untrusted server performs bootstrappings: how can the server prove, succinctly, that the ciphertexts it returns really are the prescribed bootstrappings of the ciphertexts it received, under the key material it committed to?

This note describes such a proof, built on a lattice-based succinct argument [2] whose native objects are committed vectors over a cyclotomic ring and whose native statements are *sumcheck claims* about those vectors. The exposition is aimed at a reader who knows basic ring arithmetic but not necessarily TFHE or sumcheck protocols; every object is introduced before it is used.

The contributions described here are:

1. an implementation of CGGI bootstrapping directly over the proof system’s prime modulus $q = 2^{50} - 2687$, with the parameter shape of the TFHE-rs 1.6.2 default set and noise rescaled accordingly (section 3);
2. a packing of TFHE-ring polynomials into proof-ring elements under which TFHE-ring multiplication becomes a length-16 convolution of native proof-ring multiplications (section 4);
3. an encoding of the complete blind-rotation trace, for several bootstrappings sharing one committed key, as a constant number of sumcheck claims over a single committed witness (section 5);
4. an end-to-end instantiation at the full parameters: one bootstrapping proven through the argument chain in 11.6 minutes with a 167 KB proof, the committed trace bound in strict ℓ_2 norm by a random projection added to the entry round, and the same pipeline proving and verifying four toy-parameter bootstrappings (section 6).

2 Background

2.1 CGGI bootstrapping in five objects

All ciphertexts live over $\mathbb{Z}_q$. We write p for the plaintext modulus (here $p = 32$, encoding a 2-bit message, a 2-bit carry, and a padding bit) and $\Delta = \lfloor q/p \rfloor$ for the encoding scale.

LWE. An LWE ciphertext of a message m under a binary secret $\mathbf{s} \in \{0, 1\}^n$ is a pair $(\mathbf{a}, b)$ with $\mathbf{a}$ uniform in $\mathbb{Z}_q^n$ and $b = \langle \mathbf{a}, \mathbf{s} \rangle + \Delta m + e$ for a small noise e . Decryption computes $b - \langle \mathbf{a}, \mathbf{s} \rangle$ and rounds to the nearest multiple of Δ .

GLWE. The ring variant replaces scalars by polynomials in $\mathcal{R}' = \mathbb{Z}_q[Y]/(Y^N + 1)$: a GLWE ciphertext under a key of k binary polynomials encrypts a polynomial of messages. Throughout, $k = 1$ and $N = 2048$, so a GLWE ciphertext is a pair of degree-2048 polynomials.

GGSW and the external product. A GGSW ciphertext encrypts a single bit s_i in a redundant form: $(k + 1)\ell$ GLWE rows, where row (u, j) carries $s_i B^j$ at slot u for a gadget base B and ℓ levels. Its purpose is the *external product*: given a GLWE ciphertext c , decompose each of its polynomials into ℓ balanced base- B digit polynomials, multiply each digit polynomial by the matching GGSW row, and sum. The result is a GLWE encryption of $s_i \cdot (\text{message of } c)$, with noise that grew only proportionally to B rather than to q . The digit decomposition is the entire reason GGSW exists: multiplying c directly by an encryption of s_i would scale its noise by $q/2$ in the worst case.

CMUX. The external product gives an encrypted multiplexer:

$$\text{CMUX}(\text{GGSW}(s_i), d_0, d_1) = d_0 + \text{GGSW}(s_i) \boxtimes (d_1 - d_0),$$

which equals an encryption of the message of d_0 when $s_i = 0$ and of d_1 when $s_i = 1$, without revealing s_i .

Blind rotation. Bootstrapping an LWE ciphertext $(\mathbf{a}, b)$ proceeds by encoding the lookup table as a polynomial $L \in \mathcal{R}'$ whose coefficient blocks repeat each table value ($N/16$ copies per value, with the block boundaries aligned so rounding errors land inside a block). The phase $b - \langle \mathbf{a}, \mathbf{s} \rangle$ is brought to the exponent of Y : after rescaling every component to $\tilde{a}_i = \lfloor a_i \cdot 2N/q \rfloor$, the accumulator starts as $\text{ACC} \leftarrow Y^{-\tilde{b}} \cdot L$ and runs, for $i = 1, \dots, n$,

$$\text{ACC} \leftarrow \text{CMUX}(\text{GGSW}(s_i), \text{ACC}, Y^{\tilde{a}_i} \cdot \text{ACC}),$$

so that exactly when $s_i = 1$ the accumulator is rotated by $\tilde{a}_i$ positions. After all n steps the constant coefficient of the accumulator holds $\Delta \cdot L[m]$ up to small noise, and a public rearrangement of coefficients (*sample extraction*) turns it back into an LWE ciphertext. Steps with $\tilde{a}_i = 0$ are skipped; which steps survive is public, since $\mathbf{a}$ is public.

The blind rotation is the entire cost and the entire content of the proof: $n = 918$ CMUX steps, each containing one digit decomposition and $(k + 1)^2 \ell$ polynomial multiplications in $\mathcal{R}'$.

2.2 The proof system interface

The argument system [2] commits to a vector $\mathbf{w}$ of elements of the proof ring $\mathcal{R} = \mathbb{Z}_q[X]/(X^{128} + 1)$ and proves claims of the form

$$\sum_{z \in \{0,1\}^\nu} \sum_{\text{terms } t} \gamma_t \prod_{f \in t} f(z) = v, \quad (1)$$

where each factor f is either the multilinear extension $\text{MLE}[\mathbf{w}]$ of the committed vector (the unique multilinear polynomial agreeing with $\mathbf{w}$ on the Boolean cube), the extension of a public vector, or the extension of a contiguous *segment* of $\mathbf{w}$ identified by a binary prefix of the index space. Products of factors evaluated at the same cube point z express coordinatewise relations; a degree-two term with two witness factors expresses a product of two committed values at the same position. After a sumcheck interaction, every claim reduces to evaluations of $\text{MLE}[\mathbf{w}]$ at random points, which the argument chain then proves against the commitment while also certifying an ℓ_2 bound on $\mathbf{w}$.

Two consequences shape everything that follows. First, the claim language is *coordinatewise*: it multiplies committed values at equal positions, and offers no direct way to multiply $\mathbf{w}[i]$ by $\mathbf{w}[j]$ for $i \neq j$. Second, the only smallness guarantee is an aggregate ℓ_2 norm; no per-coordinate range is proven. Both are deliberate economy choices of the underlying argument, and section 5 works within them.

3 Bootstrapping over the proof modulus

Standard TFHE works modulo 2^{64} . Proving statements about mod- 2^{64} arithmetic inside a mod- q proof system would force every equation through an encoding of wraparounds. Instead, the bootstrapping is instantiated natively modulo the proof prime $q = 2^{50} - 2687$: the same algorithms, the same parameter shape, no second modulus anywhere.

Concretely, the implementation follows the TFHE-rs 1.6.2 default parameter set `PARAM_MESSAGE_2_CARRY_2` ($n = 918$, $k = 1$, $N = 2048$) with two adaptations. Noise bounds scale by $q/2^{64}$: the LWE noise is uniform on $[-2^{31}, 2^{31}]$ and the GLWE noise on $[-2^3, 2^3]$, preserving the noise-to-modulus ratios of the original set. The gadget changes from the base- 2^{23} , one-level decomposition of TFHE-rs, which relies on power-of-two bit extraction, to an exact balanced decomposition with

$B = 2^{25}$ and $\ell = 2$, natural for a prime modulus and, as it turns out, also the proof-friendly choice: an exact decomposition is a linear identity, with no rounding remainder to carry through the constraints. The modulus switch, the redundant lookup table layout, the skip of zero steps, and the key-switching follow the reference implementation.

Polynomial multiplication in $\mathcal{R}'$ uses a partial negacyclic NTT. The 2-adic valuation of $q - 1$ is 7, so a full NTT at $N = 2048$ does not exist; six transform levels split $Y^{2048} + 1$ into 64 blocks of degree 32, and the blockwise products run on AVX-512 IFMA kernels (size-64 NTT columns and 52-bit multiply-accumulate convolution, with a final fold by $2^{50} \equiv 2687$). One multiplication takes $26 \mu\text{s}$, and a complete bootstrapping (mod switch, 918 CMUX steps, sample extraction) runs in 0.3 s single-threaded; generating the full key material and one traced bootstrapping takes 0.73 s. Correctness is tested end to end: all 16 plaintexts bootstrap through a nontrivial lookup table and decrypt correctly, both after sample extraction and after key-switching back to the small key.

4 Packing the TFHE ring into the proof ring

The proof system multiplies elements of $\mathcal{R}$, of degree 128; the trace multiplies elements of $\mathcal{R}'$, of degree 2048. The bridge is a factorization both rings share. Modulo q , the polynomial $X^{128} + 1$ splits into 64 quadratic factors $X^2 - \psi_j$, where the ψ_j are the 64 roots of $\psi^{64} = -1$; multiplication in $\mathcal{R}$ acts independently on these 64 quadratic residues (this is the incomplete NTT the proof system computes with). The same ψ_j govern the TFHE ring:

$$\mathbb{Z}_q[Y]/(Y^{2048} + 1) \cong \prod_{j=1}^{64} \mathbb{Z}_q[Y]/(Y^{32} - \psi_j) \cong \prod_{j=1}^{64} \mathbb{F}_{q^2}[Y]/(Y^{16} - x_j),$$

where the second step writes each degree-32 residue over the quadratic extension $\mathbb{F}_{q^2} = \mathbb{Z}_q[X]/(X^2 - \psi_j)$ via $Y^{16} \mapsto X$.

A polynomial $P \in \mathcal{R}'$ is therefore stored as 16 elements of $\mathcal{R}$: element t holds, in its j -th quadratic slot, the Y^t -coordinate of the j -th residue of P . Under this packing, a product in $\mathcal{R}'$ becomes

$$(P \cdot R)_t = \sum_{t_1+t_2=t} P_{t_1} \odot R_{t_2} + X \odot \sum_{t_1+t_2=t+16} P_{t_1} \odot R_{t_2}, \quad (2)$$

where $\odot$ is native multiplication in $\mathcal{R}$ and the multiplication by the public element X accounts for the wrap $Y^{16} = X$. One TFHE-ring product is thus 256 proof-ring products arranged as a length-16 convolution. The packing and identity (2) are verified in code against direct multiplication in $\mathcal{R}'$ for $N \in \{256, 2048\}$.

The point of working in this packed evaluation domain, rather than committing coefficient vectors, is that (2) contains only *ring products of stored values*: the claim language of section 2.2 can express it. A coefficient-domain encoding would instead turn each polynomial product into a dense 2048×2048 linear map interleaved with products, far outside coordinatewise reach.

5 The relation

Fix a bootstrapping and consider its surviving steps $1, \dots, S$. The committed witness contains, in the packed representation of section 4:

- the accumulator states $\text{ACC}_0, \dots, \text{ACC}_S$ ($k+1 = 2$ polynomials each);
- for each step, the digit polynomials $D_{u,j}$ of the decomposition of $Y^{\tilde{a}_i} \text{ACC}_{i-1} - \text{ACC}_{i-1}$;
- the GGSW rows $G_{u,j,v}$ of the bootstrapping key, stored once and shared by all bootstrappings in the batch;
- two *operand segments*, described below.

Packed values fill the full range of $\mathbb{Z}_q$, while the commitment is binding only for short vectors, so every stored coordinate is decomposed into eight balanced chunks of 7 bits; a value is recovered as $\sum_c 2^{7c}(\text{chunk } c)$, and all claims act on chunks through this linear recomposition. The chunk width is the same base-2⁷, eight-digit shape the argument chain uses internally for every full-range vector it commits, and for the same reasons: the digits set the norm of the committed vector, and the commitment’s binding margin degrades as that norm grows; the chain’s first fold-and-decompose step combines all witness columns within a fixed per-coefficient capacity; and the entry-round random projection of section 6 accumulates digits in 16-bit lanes, which stay in range only for digits at this scale. An earlier iteration used four 15-bit chunks, half the storage, and tripped every one of these constraints in turn.

The blind-rotation step relation splits into one linear and one quadratic family.

Recomposition and boundaries (linear). For each step and each slot coordinate, the digits must recombine to the rotated difference:

$$\sum_j B^j D_{u,j} = Y^{\tilde{a}_i} \text{ACC}_{i-1} - \text{ACC}_{i-1},$$

where multiplication by the public monomial $Y^{\tilde{a}_i}$ is, in the packed domain, a public length-16 convolution: a linear map with known coefficients. Together with the boundary equalities, $\text{ACC}_0 = \text{pack}(Y^{-\tilde{b}}L)$ and $\text{ACC}_S =$ the public output accumulator, these are all linear equations in the witness. They are batched into a single claim of the form (1): a random vector ρ , sampled from the transcript after the commitment, weights every equation, and the per-position weights are assembled into one public vector per witness segment. The claim value collects the ρ -weighted public boundary data.

Accumulation (quadratic). Each step must satisfy

$$\text{ACC}_i = \text{ACC}_{i-1} + \sum_{u,j} D_{u,j} \cdot G_{u,j,v},$$

with the products expanded by (2) into pairs $D_{u,j,t_1} \odot G_{u,j,v,t_2}$. These pairs multiply witness values at *different* positions, which the claim language cannot do directly. The remedy is operand alignment on a common index space. Write g for the multiplication group (one per step and gadget row, u, j) and order the product space by (t_1, g, t_2) . The witness carries two operand lists: **lop**, indexed by (t_1, g) and holding the digit coordinate D_{g,t_1} , and one **rop_v** per output slot, indexed by (g, t_2) and holding the key coordinate G_{g,v,t_2} . Each list has exactly one entry per distinct operand; nothing is replicated. Inside the entry sumcheck, **rop_v** is read as an ordinary segment oracle over the low (g, t_2) variables, and **lop** as a *shifted* oracle whose data variables sit above the t_2 block, in which it is constant. A product of the two at a cube point (t_1, g, t_2) is then exactly $D_{g,t_1} \odot G_{g,v,t_2}$, so all accumulation equations batch into one claim with $(\text{chunk})^2$

quadratic terms per output slot, the public weight vector carrying the equation weights and the X -twist of (2), and linear terms carrying the ACC sides. The shifted oracle’s evaluation enters the argument chain as one more opening, at the point whose challenge coordinates are the matching middle slice.

Copy claims tie the lists to their sources: a transcript-derived random vector ρ satisfies $\sum_o \rho_o \text{lop}(o) = \sum_e (\sum_{o:\text{src}(o)=e} \rho_o) D(e)$, and equality of the two random combinations binds each list to the digit (respectively key) segment. Since the lists are permutation-sized, the operand storage equals one extra copy of the digits and $k + 1$ rearrangements of the used key rows.

What the proof asserts. The claims above, plus the ℓ_2 bound certified by the chain, prove knowledge of accumulator states, digits, and key rows that satisfy every step equation, recompose correctly, have bounded aggregate norm, and connect the public input phase to the public output accumulator. Per-coordinate digit ranges are not proven; a prover could substitute a different valid decomposition of the same differences, trading noise growth for freedom that the ℓ_2 budget tolerates. The statement is correspondingly *knowledge of a bounded-norm execution* rather than of the canonical one; tightening it to the canonical trace would require range claims on the digits.

6 End-to-end instantiation

The complete pipeline commits the witness, runs the entry sumcheck over all claims, and hands the resulting evaluation claims to the argument chain of [2] as its initial openings: the two standard witness evaluations plus one opening per witness segment (twenty-five segments here: the stacked linear segment, eight digit-operand lists, and sixteen key-operand lists), padded to the next power of two.

One component of the chain is reconfigured rather than reused as-is. In its polynomial-commitment role the chain skips the random norm projection in the entry round, because there the entry witness consists of decomposition digits whose shortness the chain re-derives downstream; the extracted norm bound then carries a slack factor, which that application tolerates. Here the witness is the trace itself, and the ℓ_2 statement should bind it directly. The entry round therefore runs the chain’s coarse projection on the committed trace: a transcript-sampled random ± 1 block matrix is applied to the (signed 16-bit) witness coordinates, the image is committed alongside the other entry data, and the consistency of image and witness is checked by the next round’s sumcheck, exactly as in the chain’s interior rounds. The image does not come for free: it is $1/64$ of the witness in elements and, after its own digit decomposition, $1/16$ in committed volume, which exceeds the headroom of the standard chain geometry. Every interior round of the chain therefore runs at twice its standard height, and the final fold is reshaped to hand the last round its usual input; the last round itself is unchanged. The configuration generator now checks each round’s composed output against the next round’s capacity, a mismatch that previously failed only as a debug assertion.

At toy TFHE parameters ($N = 256$, $n = 8$) four independent bootstrappings under one committed key prove end to end: the entry sumcheck over the 2^{20} -element witness takes 155 s, the chain adds a few seconds, the proof is 167 KB, and verification takes 51 ms. At the full parameters of section 3, the trace of one bootstrapping occupies about 3.4 million committed ring elements ($2^{21.7}$), filling the middle parameter set (2^{22}). The measured run: key generation plus the traced bootstrapping in 0.73 s, witness encoding in 9.2 s, commitment in 3.5 s, the entry

sumcheck over all five claims in 649–653s across two runs, and the chain rounds in about 10s — 11.6 minutes of proving in total — yielding a 167 KB proof. Verification is measured at the toy parameters (51 ms); at the full parameters the verifier’s cost is dominated not by the chain, which is unchanged, but by re-deriving and evaluating the dense Fiat–Shamir public vectors of the entry claims, linear in their combined length of about eight million ring elements, so on the order of seconds. Two bootstrappings fit the largest parameter set (2^{24}).

Correctness of every layer is tested independently: the packing against direct TFHE-ring multiplication, the recorded traces against step-by-step replay, each claim family in isolation, the four-bootstrapping batch, and rejection of a tampered output accumulator.

7 Limitations

Four gaps separate this instantiation from a production verifiable-bootstrapping service. The digit ranges are bounded only in aggregate ℓ_2 norm, as discussed in section 5; the entry-round projection of section 6 makes that aggregate bound strict, but does not turn it into per-coordinate ranges. The proven relation covers the blind rotation, which is the only place the committed key material enters; the modulus switch, sample extraction, and key switch act on public data with public keys, so the verifier can recheck them directly. What is not proven is any well-formedness of the committed bootstrapping key itself, or its consistency with the public key-switching key: the statement is relative to the committed key material as given. The final accumulator is published in full, which reveals N valid LWE ciphertexts rather than the single extracted one; harmless under the scheme’s own security but coarser than necessary. The verifier re-derives the entry round’s Fiat–Shamir public vectors explicitly and evaluates them coordinate by coordinate; they are highly structured (scattered batching randomness and equality weights), so a succinct verifier should evaluate them lazily from that structure rather than materialize them, and until it does, verification time at full parameters is linear in the trace. And the security of the rescaled parameter set, while inheriting the noise ratios of the TFHE-rs original, has not been re-estimated against lattice attacks at the new modulus; the parameter-selection pass for both the FHE side and the argument side remains future work.

References

- [1] I. Chillotti, N. Gama, M. Georgieva, M. Izabachène. TFHE: Fast Fully Homomorphic Encryption over the Torus. *Journal of Cryptology*, 2020. Implementation: TFHE-rs 1.6.2, <https://github.com/zama-ai/tfhe-rs>.
- [2] RoKoko: Lattice-based Succinct Arguments, a Committed Refinement. 2026. Implementation: <https://github.com/lattice-arguments/rokoko>.